micro:bit

Talking About Accessibility on the micro:bit

A Practical Guide for Technology and Programming Teachers (Ages 12–16)

Why talk about accessibility with your students?

When we work with the micro:bit in the classroom, we usually focus on getting the code to run, making the project look impressive, and helping students learn to program. All of that is great. But there is a question we often forget to ask: can everyone actually use what we have created?

Accessibility is not a topic reserved for experts or large technology companies. It is a mindset we can — and should — introduce from the very first programming experiences. Designing with user diversity in mind makes our projects more robust, more creative and, above all, fairer.

This guide is designed to help you explore three big questions with your class:

1. What accessibility features does the micro:bit have, and how can we make the most of them or improve them?
2. How do we programme accessible projects from the start?
3. How do we verify that our project is genuinely accessible?

1. The micro:bit and Accessibility: What Does It Offer and What Is Missing?

The micro:bit is a development board designed to make learning to programme easier. However, like most technological devices, its accessibility has both strengths and weaknesses. Let us look at them together.

1.1. Current accessibility features

Visual outputs: the LED matrix

Both the micro:bit V1 and V2 include a 5×5 LED matrix. This matrix can display text, icons and animations. It is one of the board's clearest communication channels, although it has some limitations:

- Scrolling text can be hard to read for people with dyslexia or low vision.
- The 5×5 pixel resolution severely limits graphical representation and makes accessible typefaces impossible to display.
- There is no built-in contrast control or scroll-speed adjustment in the block-based programming environment without extra code.

Sound outputs: the built-in speaker (V2)

The micro:bit V2 includes a built-in speaker and microphone. This opens up tremendous possibilities for auditory accessibility:

- Sounds, melodies or acoustic signals can be emitted to complement or replace visual information.
- The microphone can be used as an alternative input to physical touch for people with reduced mobility.
- On the V1, sound can also be generated by connecting an external buzzer to the pins, though this requires additional hardware.

Physical inputs: buttons, pins and sensors

The micro:bit offers several ways to receive input from the user:

- Two physical buttons (A and B) that are easy to locate by touch, although they can be a barrier for people with limited fine motor skills.
- Input/output pins that allow connecting sensors, large-format switches or adapted joysticks.
- Accelerometer and compass that can serve as movement-based input alternatives for the body or a support device.
- Gesture sensor (V2): shake, tilt, free fall… very interesting possibilities for creating alternative interfaces.

Connectivity: radio and Bluetooth

Wireless communication between micro:bits opens the door to collaborative and accessible projects:

- Multiple micro:bits can work together, so one can act as an adapted controller and another as a display or actuator.
- Bluetooth allows the board to connect to external devices such as phones or tablets, which do have built-in assistive technologies (TalkBack, VoiceOver, etc.).

1.2. Limitations and how we can address them

Recognising limitations is the first step towards overcoming them. Here are the most important ones, along with some ideas for tackling them in class:

Limitation	Accessibility barrier	Possible improvement
5×5 LED display	Low resolution and contrast	Add an external OLED screen via I2C
Only 2 physical buttons	Difficult for reduced mobility	Connect large push buttons to the pins
No native text-to-speech	Not accessible for low vision	Use radio + tablet with a screen reader
Scrolling text	Hard to read with dyslexia or slow reading	Show icons or use alternative audio signals
No haptic feedback	Barrier for combined visual/hearing impairment	Connect a vibration motor to an output pin

A reflection for the classroom: when you introduce a project, ask your students "who cannot use this?" before finalising the design. That simple question is the beginning of an inclusive design mindset.

2. How to Create Accessible Projects with the micro:bit

Programming with accessibility in mind does not mean making the code more complicated — it means making it more human. In this section we will look at the key characteristics accessible code should have, what types of outputs we can use, and what extensions or extra hardware can help us.

2.1. Principles of accessible code

Multimodality: offer more than one information channel

The most important principle of accessibility is not relying on a single sensory channel. If your project shows something on the LEDs, also add an audio signal or a vibration. This benefits:

- People with low vision or blindness (who need auditory or tactile information).
- People with hearing impairment (who need visual or tactile information).
- Anyone in a noisy environment, poor lighting, or with their hands occupied.

Accessible configuration variables

A good practice is to use variables at the start of the programme to control key accessibility parameters, making them easy to adjust without touching the core logic:

- textSpeed: controls how fast text scrolls across the LED display.
- brightnessMode: allows switching between high, medium and low brightness.
- soundEnabled: allows audio alerts to be turned on or off.
- displayTime: defines how many seconds an icon is shown before disappearing.

Readable and commented code

Accessibility also includes making the code itself accessible for anyone who reads or modifies it. Well-commented code helps other people (classmates, parents, students with cognitive diversity) understand what each part does. Some tips:

- Use descriptive variable names: temperature instead of t, or buttonPressed instead of bp.
- Add comments to each block or function explaining its purpose.
- Break the code into functions with clear names: showResult(), playAlarm(), etc.

Managing alternative inputs

Not all users can interact in the same way. Accessible code provides multiple ways to trigger the same function:

- Button A, Button B and the A+B combination as three separate inputs that trigger the same function (control redundancy).
- Gestures detected by the accelerometer (shake, tilt) as alternatives to button presses.

- Sound level (microphone in V2) to trigger actions without any physical contact.

2.2. Types of outputs for inclusive design

Let us look in detail at how to make the most of each output type to create more accessible projects:

Enriched visual output

- Prefer icons to text whenever possible: they are processed faster and avoid reading barriers.
- Use the led.setBrightness() function to adjust brightness based on the environment or the user's needs.
- Connecting an external OLED display allows you to show large text with better contrast.
- To indicate states, use different blinking patterns (fast = urgent, slow = normal) rather than just different icons.

Audio output

- Use different notes or melodies to distinguish states: a low note for 'error' and a high note for 'correct'.
- Vary the duration of tones to convey information (similar to an adapted Morse code).
- Always provide an option to mute sound: not every environment allows audio.

Tactile and physical output

- Connect a vibration motor to an output pin to create haptic feedback — very useful for people with simultaneous visual and hearing impairment.
- A servo motor rotating to different positions can represent information in a physical and tactile way.
- External NeoPixel LEDs (RGB LED strips) allow colours and patterns that the 5×5 display cannot reproduce, improving visual perception.

2.3. Useful extensions and tools

MakeCode, the micro:bit's main programming environment, allows extensions to be added that expand the accessible possibilities of any project:

Extension / Hardware	What it contributes to accessibility	Ideal for...
NeoPixel (extension)	External RGB LEDs with a wide range of colours and adjustable brightness	People with low vision or colour blindness
Servo / DC Motor	Physical and tactile output to represent data	People with visual impairment

Extension / Hardware	What it contributes to accessibility	Ideal for...
OLED I2C display	Large, clear text on a dedicated screen	People with low vision or dyslexia
Capacitive touch sensor	Large, sensitive touch inputs	Reduced hand mobility
Joystick:bit / Kitronik	Alternative control with a physical joystick	Fine motor difficulties
Bluetooth extension	Connection to tablets or phones with assistive technologies	Screen reader users

Practical tip: you do not need to use everything at once. Adding just one or two accessibility features already makes a project significantly more inclusive. Challenge your students to add 'at least one accessibility improvement' to every project they build.

2.4. Universal Design for Learning (UDL) applied to programming

The UDL framework invites us to think in three dimensions when designing learning experiences, and we can apply these directly to micro:bit project design:

- Multiple means of representation: show information in more than one way (visual + audio + tactile).
- Multiple means of action and expression: allow interaction with the project in more than one way (buttons, gestures, sound).
- Multiple means of engagement: the project can be adjusted to different difficulty levels through configurable variables.

3. Accessibility Testing: How Do I Know My Project Works for Everyone?

Programming with accessibility in mind is only the first step. The next is verifying that those design decisions actually work. Accessibility testing is a part of the development process that gets very little attention in the classroom, yet it can be one of the most enriching and empathy-building activities we do with our students.

3.1. Code-level testing

Multimodality review

Before running the code, ask your students to review it using this checklist:

- Is all important information communicated through at least two different channels?
- Is there an option to mute the audio without losing information?
- Can the main function be triggered in more than one way?
- Is there a minimum reaction time that could be a barrier? Can it be adjusted?

Simulation in MakeCode

The MakeCode simulator is a very useful tool for quick testing without needing the physical board. For accessibility testing in the simulator:

- Test the speed of scrolling text: can it be read comfortably, or does it move too fast?
- Activate the simulator's buttons and gestures to verify all alternative inputs work correctly.
- Listen to the audio in the simulator and check that the sounds are clearly distinguishable from each other.
- Simulate operation with the radio enabled if the project involves multiple devices.

Peer code review with an accessibility reviewer role

A highly recommended activity is pair code review where one student explicitly takes the 'accessibility role'. That person must answer:

- Could someone who cannot see the LED display use this project?
- Could someone who cannot hear the sounds use it?
- Could someone who can only use one hand, or who has motor difficulties, use it?
- Is the code understandable for someone reading it for the first time?

3.2. Testing with the physical micro:bit

The 'eyes closed' test

This test is simple, visually engaging and generates a great deal of empathetic learning in the classroom. Students use their project with eyes closed for 2–3 minutes and try to answer:

- Can I tell what state the project is in using only sound or vibration?
- Can I find the buttons and use them without seeing the board?
- Is the auditory feedback clear enough to understand what is happening?

The 'ears covered' test

The complementary version: using the micro:bit with earplugs or in a loud environment (loud music, a nearby fan), and checking:

- Is the visual information on the LED display enough to understand the project's state?
- Are the icons clear enough without any supporting text?
- Does the blinking pattern or LED animation communicate urgency or normality in an understandable way?

The 'one hand only' test

Place the micro:bit on a table and ask students to interact with it using only one hand (or just two fingers). Observe:

- Is it possible to activate all the project's functions with limited movement?
- Is the A+B button combination required for any critical function? If so, is there an alternative?
- Can the accelerometer gestures be performed without needing two hands?

Testing with real users

The best accessibility test is always to try the project with a diverse range of people. Some ideas for the classroom:

- Group swap: Group A tests Group B's project with no prior instructions. Any confusion that arises is an accessibility barrier.
- Testing with people outside the project (family members, teachers from other subjects) who can provide an unbiased perspective.
- Where possible, collaborate with the school's inclusion or guidance team so that students with specific needs can act as user testers for projects, always with consent and in a safe environment.
- Record test observations in a simple table: what function was being performed? What difficulty appeared? What improvement is proposed?

3.3. Accessibility checklist for micro:bit projects

To finish, here is a checklist you can use as a rubric or as a review guide with your students. It can be printed and used on paper during the testing phase:

#	Accessibility criterion	Met? ✓ / ✗ / ⚠
1	Information is communicated through at least 2 channels (visual + auditory, etc.)	

#	Accessibility criterion	Met? ✓ / ✗ / ⚠
2	There is an option to mute sound without losing information	
3	Main functions can be activated in at least 2 different ways	
4	Scrolling text has an adjustable or sufficiently slow speed	
5	Icons are recognisable without needing to read text	
6	The project works with one hand only	
7	Accelerometer gestures are gentle and require no excessive physical effort	
8	The code uses descriptive variable names	
9	The code has comments explaining each main block	
10	The eyes-closed test has been completed with a positive result	
11	The ears-covered test has been completed with a positive result	
12	The project has been tested by someone outside the development team	
13	Difficulties encountered during testing have been recorded and analysed	

To Close: The Classroom as a Laboratory of Inclusion

Working on accessibility with the micro:bit in the classroom is much more than adding a buzzer or an extra LED. It is an opportunity for students to develop empathy, to think about people when designing technology, and to learn that good code is not just code that works — it is code that works for everyone.

You do not need to reach complex projects from day one. Simply introducing a question at the end of each session — "who cannot use this, and why?" — is already planting a very valuable seed.

Accessible technology is not special technology. It is simply good technology.